

MEDTRACK -AI DRIVEN PATIENT MONITORING SYSTEM

PROJECT REPORT

Submitted in fulfilment of the requirements for the Course Project of

SWE2013 – Advanced Java Programming

in

M.Tech (SE)

by

<PAWAR POSHITHA BAI>

Under the guidance of

Dr. Mareeswari V

SCORE



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science Engineering and Information Systems

Nov 2025

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1	ABSTRACT	3
2	Introduction	3-4
3	Problem Statement	4-5
4	Literature Review	5-7
5	System Design	8-11
6	Software Requirement Specifications	12-21
7	Implementation	
	7.1 Source code snippet	21-37
	7.2 Screenshots with explanation	37-48
8	Performance Analysis and Discussion	48-51
9	Conclusion	50
10	Future Enhancements	50-51

11	References	51-52
----	------------	-------

1.ABSTRACT:

The MedTrack: AI-Driven Patient Monitoring System is an intelligent healthcare solution designed to enhance patient care in rural hospital networks by providing real-time health monitoring and predictive alerts for critical conditions. The system integrates wearable health data with a scalable backend built using Spring Boot, RESTful APIs, and JDBC, enabling efficient data storage and retrieval. By employing a Model-View-Controller (MVC) architecture, the system ensures modularity, maintainability, and clear separation of concerns across components.

A machine learning model is integrated to perform anomaly detection, allowing early identification of irregular patient vitals and alerting healthcare professionals through automated notifications. Role-based dashboards are provided for doctors, nurses, and administrators, ensuring that relevant health insights and analytics are accessible based on user privileges. The platform aims to reduce response time in medical emergencies and improve healthcare delivery in resource-limited environments.

2.INTRODUCTION:

2.1 Project Overview

The **MedTrack: AI-Driven Patient Monitoring System** is a real-time ICU monitoring platform developed using **Spring Boot, RESTful APIs, JDBC**, and follows the **Model-View-Controller (MVC)** architecture. The system continuously receives vital-sign data from ICU medical devices and processes it using a trained **machine learning anomaly detection model**. When abnormal patterns are detected, the system automatically generates alerts, stores them in a secure SQL database, and displays them on a centralized dashboard for doctors and nurses.

MedTrack transforms traditional ICU monitoring into an intelligent, automated framework that improves patient safety, reduces manual workload, and ensures faster clinical intervention.

2.2 Significance of the Project

The MedTrack system provides major benefits for both medical staff and hospital administration:

- **For Medical Staff:**

- Ensures continuous, automated monitoring of ICU patients.
- Provides instant alerts when abnormal vitals are detected.
- Enables faster clinical response by displaying patient details (room, bed, timestamp) on a dashboard.
- **For Hospital Administration:**
- Reduces dependency on manual observation and isolated device alerts.
- Creates structured digital records of patient alerts for audit and analysis.
- Supports better ICU management through data-driven insights and early anomaly detection.

2.3 Key Features

The core functionality of MedTrack focuses on automated anomaly detection, reliable alert delivery, and real-time visualization.

- **Real-Time ICU Data Monitoring:** Continuously processes heart rate, BP, SpO₂, temperature, and other vitals.
- **Machine Learning–Based Anomaly Detection:** Classifies patient data as normal or critical using a trained ML model.
- **RESTful Alert Communication:** Sends alert data to the backend through a secure POST API (/api/alerts).
- **Centralized Dashboard:** Displays alerts with patient identity, room number, and timestamp for medical staff.
- **MVC Architecture:** Clean separation of concerns—Model (entities), Controller (REST endpoints), View (UI dashboards).
- **Secure Data Storage using JDBC:** Stores patients and alert logs in an SQL database.
- **Role-Based Access (Optional):** Doctors, nurses, and admins can access different dashboards.

3.PROBLEM STATEMENT:

3.1 Background

Monitoring human vital signs—such as heart rate, blood pressure, oxygen saturation, body temperature, and glucose levels—is essential for identifying potential health risks at an early stage. Traditionally, the interpretation of these vital parameters has depended heavily on manual

assessment by healthcare professionals. This manual process is prone to delay, inconsistency, and human error, especially in large-scale or remote healthcare settings.

With the rising demand for rapid health assessment systems, there is an urgent need for digital and automated solutions that can analyze vital signs in real time. Modern healthcare environments expect intelligent, data-driven systems capable of providing early warnings, predicting abnormal conditions, and ensuring timely intervention. Current conventional methods fall short in delivering immediate, consistent, and scalable health-risk predictions.

3.2 Identified Problems:

- The lack of an automated health condition prediction system leads to several critical issues:
- Delayed detection of abnormal vital signs, resulting in late medical attention and increased health risks.
- Inconsistent manual readings, which may vary between healthcare professionals and increase chances of misinterpretation.
- Inability to process large volumes of patient data, preventing timely analysis during emergencies or mass screenings.
- No predictive insights, meaning abnormalities are addressed only after they occur instead of preventing them.
- Class imbalance in datasets, causing machine learning models to misclassify abnormal cases without proper handling techniques like SMOTE.

3.3 Need for the System:

The ML-based Health Condition Prediction System is essential to enable accurate, fast, and scalable analysis of human vital signs. By using Machine Learning algorithms such as Random Forest, Logistic Regression, and SVM, the system automates the classification of normal and abnormal conditions based on real-time data.

The integration of SMOTE for data balancing, performance metrics, and model comparison strengthens prediction quality and ensures fairness across all health categories. This digital solution enhances reliability, supports early intervention, and contributes to improved healthcare outcomes by enabling data-driven decision-making and continuous monitoring of patient vitals.

4.LITERATURE REVIEW:

S.No	Author / Team	Title of Paper	Source / Year	Methods Used	Merits & Demerits Summary
1	N. Ayub	Implementing a Java Microservice for Credit Fraud	ResearchGate, 2024	Java microservices, ML classifiers	Strong microservice architecture; not healthcare-specific but backend

		Detection Using ML			architecture is highly reusable.
2	Hassan & Lee	Intelligent NICU Monitoring: Real-Time Anomaly Detection	AICCC Conf., 2024	CNN, LSTM, deep learning on vital signs	High accuracy for NICU vital signs; requires GPU and large datasets.
3	IEEE Healthcare Team	Isolation Forest Anomaly Detection in Vital Signs	IEEE Xplore, 2023	Isolation Forest, unsupervised ML	Detects unseen anomalies well; lacks REST API alert mechanism.
4	R. Kapoor & W. Lin	Anomaly Detection in Wearables Data: Framework Review	PMC, 2022	IoT signals, anomaly detection frameworks	Strong conceptual basis; lacks endtoend ML + backend implementation.

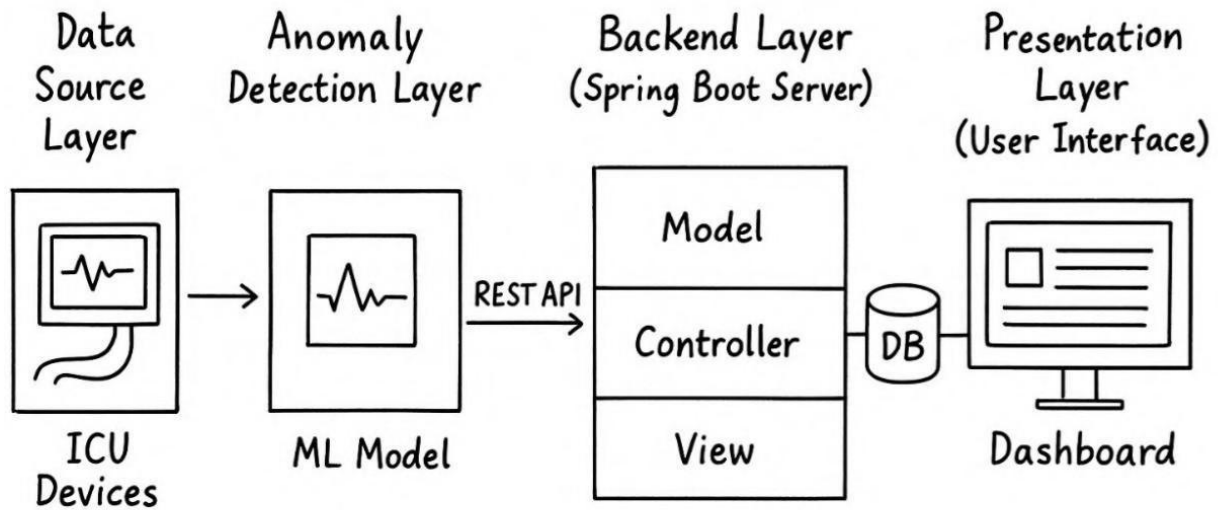
5	IJCET Team	Machine Learning-Based Vital Signs Analysis	IJCET, 2023	ML classification, health datasets	Good ML comparison; doesn't include real-time alerting system.
6	Spring Boot Dev Team	Efficient Backend Development with Spring Boot	Scribd, 2024	Spring Boot APIs, REST services	Excellent backend patterns; unrelated to medical domain but useful for system design.
7	A. Ojha et al.	Real-Time Fraud Analytics via AI Cloud Microservices	ResearchGate, 2024	Cloud microservices, asynchronous alerts	Highly scalable architecture; not applied to healthcare.

8	S. Kang & H. Woo	LSTM Autoencoder-Based ICU	IEEE Xplore, 2022	LSTM Autoencoder,	High anomaly sensitivity;
		Anomaly Detection		ICU timeseries	computationally expensive.
9	Yoon et al.	Vital Sign Prediction Using IoT Wearables	ScienceDirect, 2023	IoT sensors, ML regression	Strong IoT workflow; lacks ML risk classification.
10	WHO Medical Division	Early Warning System from Vital Sign Thresholds	WHO, 2023	Medical thresholds: HR, BP, SpO ₂	Clinically reliable; manual thresholds, not ML-powered.

11	Patel & Ganesan	ML Algorithm Comparison for Diagnosis	Springer, 2023	Random Forest, SVM, LR	Shows RF high accuracy; does not address alerting workflow.
12	Kaggle Contributors	Human Vital Sign Dataset (130k Records)	Kaggle, 2024	Feature extraction, ML modelling	Large dataset; lacks labeled risk categories (needs preprocessing).
13	Google FitBit Research	Wearable Stress & Heart-Rate Anomaly Detection	Google Research, 2022	Wearable sensors, anomaly detection	Very accurate signals; proprietary hardware.
14	MIT ML Lab	Medical Time Series Forecasting using ML	MIT Press, 2023	Time-series ML, prediction models	Useful for forecasting deterioration; high training complexity.
15	Lucas et al.	Review of ML for Real-Time Health Monitoring	Springer, 2024	ML reviews, IoT integration	Comprehensive survey; lacks full system implementation.

5.SYSTEM DESIGN:

5.1 SYSTEM ARCHITECTURE AND OVERVIEW



The proposed MedTrack: AI-Driven Patient Monitoring System, uses a layered structure. It combines real-time data collection, machine learning for spotting unusual patterns, and RESTful communication among the system components. The structure follows the Model-View-Controller (MVC) pattern to promote modularity, scalability, and simple maintenance.

SYSTEM OVERVIEW:

This system contains four primary layers:

Data Source Layer:

This layer includes ICU patient monitoring devices that continuously capture vital parameters such as heart rate, oxygen saturation (SpO₂), and blood pressure. These data streams are transmitted to the anomaly detection module.

Anomaly Detection Layer:

The machine learning model processes real-time data to identify abnormal health conditions. When an anomaly is detected, the model generates an alert and sends it to the backend through a RESTful POST request to the endpoint `/api/alerts`.

Backend Layer (Spring Boot Server):

The backend uses Spring Boot and follows the MVC structure.

- **Model:** Represents entities like Patient and Alert.
- **View:** Displays dashboards for doctors and nurses, showing alerts and patient information.
- **Controller:** The AlertController manages creating, retrieving, and deleting alerts. It uses AlertRepository and PatientRepository for database interaction via JDBC.

Presentation Layer (User Interface):

The front-end dashboard shows real-time alerts, including patient name, room number, and timestamp. Authorized medical staff can quickly view and manage these alerts.

Data Flow:

- Patient data is continuously collected from ICU monitoring devices.
- The ML module analyses the data and detects anomalies.
- Detected anomalies trigger an alert sent via the **REST API** to the Alert Controller.Alert Controller saves the data in backend (SQL). The data will be retrieved from database and sent it to the respective pages
- The dashboard displays updated alerts, allowing quick medical response

5.2 WORKFLOW DESCRIPTION:

The operation of the MedTrack: AI-Driven ICU Monitoring System follows a secure and continuous processing workflow to ensure that patient vitals are monitored in real time and abnormalities are detected at the earliest point.

1. Data Acquisition:

ICU monitoring devices continuously capture vital signs such as heart rate, blood pressure, oxygen saturation, temperature, and respiration rate. These values are transmitted to the system backend (see architecture image).

2. Backend Processing:

The Spring Boot server receives raw vital data through REST APIs and validates the input. The data is then stored securely in the patient database using JDBC as the persistence layer.

3. Machine Learning Prediction:

The Service Layer forwards the incoming vitals to the integrated ML prediction module. The trained model analyzes the parameters and classifies the patient condition as Normal

or Abnormal. If abnormal readings are detected, a prediction alert is immediately generated.

4. **Alert & Dashboard Update:**

The system sends instant notifications to authorized medical staff. The updated health status, along with anomaly details, is displayed on the dashboard for doctors and ICU administrator.

5. **Continuous Monitoring Loop:**

As new vitals arrive, the system repeats the cycle—storing new readings, generating predictions, updating dashboards, and sending alerts—thus creating a fully automated monitoring loop.

5.3 Data Flow Design

- **Input:**

Continuous real-time streams of patient vital signs (HR, BP, SpO₂, temperature, respiration rate).

Doctor/Admin inputs such as login credentials, patient registration details, and manual remarks if required.

- **Processing:**

- The Controller Layer receives data via REST endpoints.

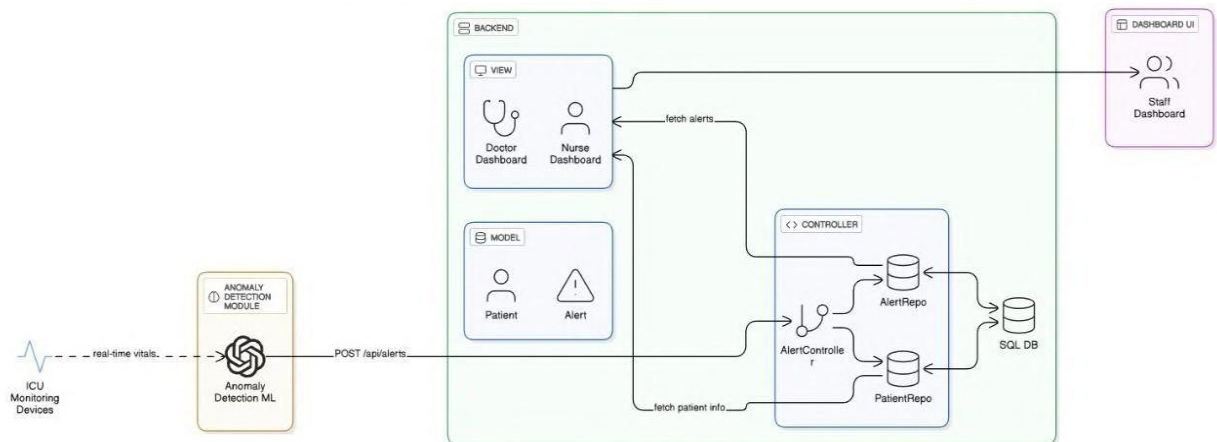
- The Service Layer handles business logic:

- Vital data validation
- Database operations using JDBC
- Forwarding vitals to the ML model for prediction

- The ML engine performs abnormality detection and returns prediction results.

- **Output:**

- Real-time **alerts** for abnormal conditions.
- Updated patient condition status (Normal/Abnormal) on dashboards.
- Historical charts, logs, and analytics views for medical staff.
- Secure storage of predictions and vital datasets for further analysis.



5.4 Technology Stack

COMPONENTS	Technology Used
Machine Learning Model	Python, Random Forest, Pandas, NumPy, smote
Model Storage	Joblib (.pkl model)
Alert Generation	Python Requests (REST API call)
Backend Framework	Spring Boot (Java)
Database	MySQL Database
Connectivity	REST API (HTTP), Spring Data JPA, Hibernate
Frontend/View Layer	HTML, CSS, JavaScript (Dashboard)
Visualization	Simple tables / charts (optional: Chart.js)
Security	Spring by using role based Security (Authentication & Authorization)
IDE / Development Tools	VS Code (Python), Netbeans (Java), MySQL
Programming Languages	Python 3.13.9, Java (JDK 23)

Other tools	Kaggle Dataset, Browser (Chrome) – to view the dashboard through local server, Build tool-Maven.
-------------	--

6.SOFTWARE REQUIREMENT SPECIFICATIONS:

MEDTRACK -AI DRIVEN PATIENT MONITORING SYSTEM

6.1 Introduction

6.1.1 Purpose

The purpose of this SRS document is to describe the functional and non-functional requirements of the AI-Based Patient Health Risk Prediction and Real-Time Alert Management System. The system predicts patient health risks using machine learning models based on vital signs and sends real-time alerts to healthcare providers through a backend service. This SRS defines the scope, features, interfaces, and constraints for developers, testers, and stakeholders.

6.1.2 Scope

The proposed system combines machine learning, IoT inputs, a backend API, and a monitoring dashboard to automatically classify patient risk levels and trigger alerts. The scope includes:

- Collecting patient vitals (heart rate, BP, temperature, SpO₂).
- Predicting risk category using a trained ML model.
- Sending alerts to backend via REST API.
- Storing and displaying alerts to medical staff.
- Providing patient lookup and alert clearance functions.

The system improves response time, reduces manual workload, and supports proactive clinical care.

6.1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
ML	Machine Learning
API	Application Programming Interface

IoT	Internet of Things
RF	Random Forest Classifier
CRUD	Create, Read, Update, Delete

6.1.4 References

Kaggle Human Vital Sign Dataset

Python Scikit-Learn Documentation

Spring Boot REST API Documentation

IEEE SRS 830-1998 Standard

6.2. Overall Description

6.2.1 Product Perspective

The system is an independent AI-based decision-support tool to be integrated with hospital monitoring systems. It contains:

- Python predictive model
- Spring Boot backend alert service
- Database for storing alerts and patient info
- Dashboard for staff

6.2.2 Product Functions

1. Accept vital signs as input (live or dataset).
2. Predict risk category (Normal / Critical).
3. Generate alerts for critical cases.
4. Send alert to backend via HTTP POST.
5. Store alerts with timestamp and patient ID.
6. Display alerts and patient details via dashboard.
7. Allow deletion of handled alerts.

6.2.3 User Characteristics

User Type Characteristics

Doctors Basic system usage, reviews alerts

User Type Characteristics

Nurses Monitor dashboard and respond to alerts

Admin Manage patients, alerts, backend operations

Developers Maintain ML model & backend

6.2.4 Constraints

- Dependent on accurate vital data for ML prediction.
- Requires stable network for IoT-to-backend communication.
- Server must be running for alerts to be received.
- Dataset biases may affect model accuracy.

6.2.5 Assumptions and Dependencies

- All patient IDs exist in the backend database.
- Data preprocessing is correctly performed before prediction.
- IoT hardware is operational (if integrated).
- Model (.pkl) file is correctly loaded during runtime.

6.3. System Features

6.3.1 Feature: Risk Prediction Module

Description

Uses Random Forest classifier to predict patient health condition.

Inputs

- Heart Rate

-
-

Blood Pressure

Oxygen Saturation

- Body Temperature

Outputs

- “Normal” or “Critical”

6.3.2 Functional Requirements

1. Patient Data & Vital Sign Processing

- Ability to input vital signs such as:
 - Heart Rate
 - Systolic/Diastolic Blood Pressure
 - Oxygen Saturation
 - Body Temperature
- Automatic preprocessing and model-ready formatting of input data.
- Support for batch predictions (dataset-based) or real-time input.

2. Machine Learning Prediction Module

- System must load a trained Random Forest .pkl model.
- The ML model shall classify patient state into:
 - **Normal**
 - **Critical**
- System shall provide prediction metrics including Accuracy, Precision, Recall, and F1Score.
- Predictions must be generated within 1 second.

3. Alert Generation (Python)

- For **Critical** predictions, the system must create an alert message.
- Alerts must include:

-
-

- Patient ID ○ Risk status ○

- Vital abnormalities detected ○

- Timestamp

Alerts shall be sent to the backend via REST API (/api/alerts).

System must handle exceptions for API failure, network issues, or missing patient data.

4. Backend Alert Management (Spring Boot)

- Backend must validate patient ID using database records.
- Alerts must be stored in the database with:
 - Message ○ Timestamp
 - Patient details
- Backend must provide the following endpoints:
 - **POST /api/alerts** → receive and store alerts ○
 - GET /api/alerts** → retrieve all active alerts ○
 - DELETE /api/alerts/{id}** → remove resolved alerts

5. Patient & Staff Dashboard

- Display all alerts with patient name, room no., bed no., doctor ID, and nurse ID.
- Show real-time list of active alerts.
- Option to clear/delete alerts once handled.
- Provide search and filter functionality by:
 - Patient ID ○
 - Severity ○

-
-

Time o

Doctor/Nurs

e

6. System Notifications

- Backend shall provide backend-level alert confirmations.

7. Modular Expansion

Easy integration of IoT devices for live vital sign input.

Support for replacing or upgrading the .pkl model without modifying backend logic.

- Support for adding new prediction features (ECG, respiratory rate, etc.) in future releases.

6.3.3 Non-Functional Requirements

Category | Requirement

Performance

- System must handle up to **1000 prediction/alert operations per hour**.
- API response time for alert submission must be less than **2 seconds**.
- ML prediction time must not exceed **1 second**.

Security

- Sensitive patient data must be protected using secure backend validation.
- Communication between Python and backend should support HTTPS (future upgrade).
- Backend must enforce role-based access (Doctor, Nurse, Admin).
- No unauthorized user should access patient alert information.

Scalability

-
-
- Both ML model and backend should support horizontal scaling.
- Database should support large alert logs and patient records over time.
- System must allow future integration with cloud-hosted services. **Reliability**
- System uptime $\geq 99\%$.
- Alerts must never be lost; if API fails, retry mechanism will handle it.
- Database should have scheduled backups.

Usability

- Alert dashboard must be easy to interpret by medical staff with minimal training.
- Critical alerts should be visually highlighted.
- UI should be responsive and work on desktop and mobile screens.

Maintainability

- Code must follow modular architecture: ○ Python ML Module ○ Java Spring Boot Backend ○ Repository-Service-Controller pattern
- Model file (.pkl) should be replaceable without code modifications.

Availability

- System should be operational 24/7.
- Maintenance windows allowed during low hospital load.

Portability

- Backend should run on Windows, Linux, or cloud servers.
- Python scripts must run on any system with Python 3+ installed.
- Works on major browsers (Chrome, Edge, Firefox).

6.4 System Interfaces

6.4.1 User Interface (UI)

- Web dashboard built using HTML, CSS, JavaScript.
- Displays active critical alerts and patient details.
- Allows staff to clear resolved alerts.

6.4.2 Backend Interface (Application Logic)

- Spring Boot REST API
 - handles: ○ Alert saving ○ Patient validation ○ Alert retrieval
 - Deletion endpoints
- Backend manages timestamps, database transactions, and security.

6.4.3 Machine Learning Interface

- Python loads .pkl model using Joblib.
- Generates predictions for each patient.
- Sends alerts to backend using JSON via requests.post().

6.4.4 Database Interface

- MySQL or H2 stores:
 - Patient details
 - Alert logs
 - Timestamped records
- Accessed via Spring Data JPA.

6.5 Software Requirements

Frontend

- HTML
- CSS
- JavaScript

Backend

- Java (JDK 17+)
- Spring Boot
- Spring Data JPA
- Spring Security (optional)
- REST API

Machine Learning

- Python 3.13.9
- Pandas, NumPy
- Scikit-Learn
- Joblib
- Requests library

Database

- MySQL / H2 Database

Tools

- IDE: VS Code, Neatbeans
- Build Tool: Maven

6.6 Hardware Requirements:

Component	Minimum Requirement
Processor	Intel i5 / AMD Ryzen 5
RAM	8 GB
Storage	50 GB free space
Operating System	Windows 10 / Linux Ubuntu 20.04+
Web Server	Embedded Tomcat (Spring Boot)
Network	Stable LAN/Wi-Fi connection

6.7. Future Enhancements

- Deep learning models (LSTM, CNN)
- Cloud-based deployment • Mobile app for doctors
- Wearable IoT integration
- Predictive trend analysis

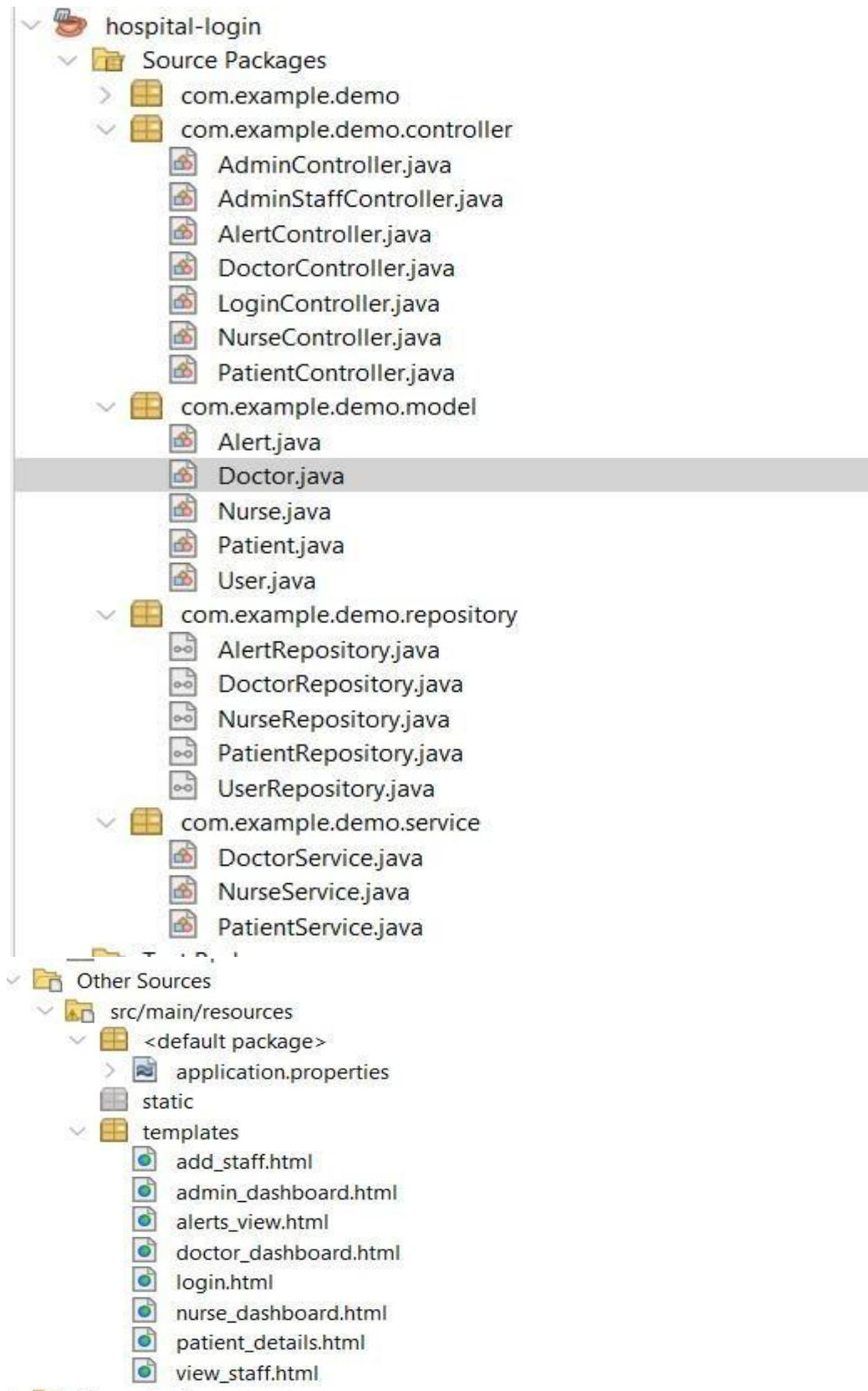
6.8. Conclusion

This SRS defines the complete requirements for a machine learning-based patient monitoring and alert system designed to improve clinical decision-making and ensure timely interventions.

7.IMPLEMENTATION:

7.1 Source code snippet Project

Structure:



Pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>hospital-login</artifactId>
  <version>1.0.0</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
  </parent>

  <dependencies>
    <!-- Spring Boot Test dependencies -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-thymeleaf</artifactId>
  </dependency>
  <dependency>
```

Model -

Alert.java

```
package com.example.demo.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "alerts")
public class Alert {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String patientId;
    private String message;
    private LocalDateTime timestamp;

    // Getters and setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getPatientId() { return patientId; }
    public void setPatientId(String patientId) { this.patientId = patientId; }

    public String getMessage() { return message; }
    public void setMessage(String message) { this.message = message; }

    public LocalDateTime getTimestamp() { return timestamp; }
    public void setTimestamp(LocalDateTime timestamp) { this.timestamp = timestamp; }
}
```

Doctor.java

```

package com.example.demo.model;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "doctors") // ✓ exact table name in DB
public class Doctor {

    @Id
    private String doctor_id;

    private String doctor_name;
    private String specialization;
    private String phone_no;
    private String email;
    private Integer room_no;
    private String shift_time;

    // ✓ Getters & Setters
    public String getDoctor_id() { return doctor_id; }
    public void setDoctor_id(String doctor_id) { this.doctor_id = doctor_id; }

    public String getDoctor_name() { return doctor_name; }
    public void setDoctor_name(String doctor_name) { this.doctor_name = doctor_name; }

    public String getSpecialization() { return specialization; }
    public void setSpecialization(String specialization) { this.specialization = specialization; }

    public String getPhone_no() { return phone_no; }
    public void setPhone_no(String phone_no) { this.phone_no = phone_no; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Integer getRoom_no() { return room_no; }
    public void setRoom_no(Integer room_no) { this.room_no = room_no; }

    public String getShift_time() { return shift_time; }
    public void setShift_time(String shift_time) { this.shift_time = shift_time; }
}

```

Controller-
Alertcontroller.java

```

package com.example.demo.controller;

import com.example.demo.model.Alert;
import com.example.demo.model.Patient;
import com.example.demo.repository.AlertRepository;
import com.example.demo.repository.PatientRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.time.LocalDateTime;
import java.util.*;

@RestController
@RequestMapping("/api/alerts")
public class AlertController {

    @Autowired
    private AlertRepository alertRepo;

    @Autowired
    private PatientRepository patientRepo;

    // POST /api/alerts — receives alert from IoT or service
    @PostMapping
    public ResponseEntity<?> receiveAlert(@RequestBody Map<String, String> alertData) {
        String patientId = alertData.get("patientId");
        String message = alertData.get("message");

        Optional<Patient> patientOpt = patientRepo.findById(patientId);
        if (patientOpt.isEmpty()) {
            return ResponseEntity.badRequest().body("X Patient not found for ID: " + patientId);
        }
    }
}

```

```

        Alert alert = new Alert();
        alert.setPatientId(patientId);
        alert.setMessage(message);
        alert.setTimestamp(LocalDateTime.now());
        alertRepo.save(alert);

        return ResponseEntity.ok("✓ Alert stored for patient " + patientId);
    }

    // GET /api/alerts — returns all alerts with patient info
    @GetMapping
    public List<Map<String, Object>> getAllAlerts() {
        List<Map<String, Object>> alertList = new ArrayList<>();

        for (Alert alert : alertRepo.findAll()) {
            Map<String, Object> map = new LinkedHashMap<>();
            map.put("patientId", alert.getPatientId());
            map.put("message", alert.getMessage());
            map.put("timestamp", alert.getTimestamp().toString());

            patientRepo.findById(alert.getPatientId()).ifPresent(p -> {
                map.put("patientName", p.getPatient_name());
                map.put("roomNo", p.getRoom_no());
                map.put("bedNo", p.getBed_no());
                map.put("doctorId", p.getDoctor_id());
                map.put("nurseId", p.getNurse_id());
            });

            alertList.add(map);
        }

        return alertList;
    }

    // Delete alert by ID
    @DeleteMapping("/{id}")
    public void deleteAlert(@PathVariable Long id) {
        alertRepo.deleteById(id);
    }

    public List<Alert> getAlerts() {
        return alertRepo.findAll();
    }
}

```

AdminController.java

```
package com.example.demo.controller;

import com.example.demo.model.Doctor;
import com.example.demo.model.Nurse;
import com.example.demo.model.Patient;
import com.example.demo.service.DoctorService;
import com.example.demo.service.NurseService;
import com.example.demo.service.PatientService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;

import java.util.List;

@Controller
public class AdminController {

    @Autowired
    private DoctorService doctorService;

    @Autowired
    private NurseService nurseService;

    @Autowired
    private PatientService patientService;

    // Show Admin Dashboard
    @GetMapping("/admin/dashboard")
    public String showDashboard() {
        return "admin_dashboard";
    }
}
```

```

// Navigate to Add Staff Page
@GetMapping("/admin/addStaff")
public String addStaff() {
    return "add_staff";
}

// Navigate to View Staff Page
@GetMapping("/admin/viewStaff")
public String viewStaff(Model model) {
    List<Doctor> doctors = doctorService.getAllDoctors();
    List<Nurse> nurses = nurseService.getAllNurses();
    List<Patient> patients = patientService.getAllPatients();

    model.addAttribute("doctors", doctors);
    model.addAttribute("nurses", nurses);
    model.addAttribute("patients", patients);

    return "view_staff";
}
}

```

Repository-

AlertRepository.java

```

package com.example.demo.repository;

import com.example.demo.model.Alert;
import org.springframework.data.jpa.repository.JpaRepository;

public interface AlertRepository extends JpaRepository<Alert, Long> {
}

```

Service-

DoctorService.java

```
package com.example.demo.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.demo.model.Doctor;
import com.example.demo.repository.DoctorRepository;
import java.util.List;

@Service
public class DoctorService {
    @Autowired
    private DoctorRepository doctorRepository;

    public void saveDoctor(Doctor doctor) {
        doctorRepository.save(doctor);
    }

    public List<Doctor> getAllDoctors() {
        return doctorRepository.findAll();
    }
}
```

NurseService.java

```
package com.example.demo.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.demo.model.Nurse;
import com.example.demo.repository.NurseRepository;
import java.util.List;

@Service
public class NurseService {
    @Autowired
    private NurseRepository nurseRepository;

    public void saveNurse(Nurse nurse) {
        nurseRepository.save(nurse);
    }

    public List<Nurse> getAllNurses() {
        return nurseRepository.findAll();
    }
}
```

View-

Admin_dashboard.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Admin Dashboard</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">

  <style>
    body {
      background-color: #f4f6f8;
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      margin: 0;
    }

    /* Navbar */
    .navbar {
      background-color: #34495e;
      color: white;
      padding: 10px 20px;
      position: fixed;
      width: 100%;
      top: 0;
      z-index: 1000;
    }

    .navbar-brand {
      color: white;
      font-weight: bold;
    }
  
```

```

    /* Sidebar */
    .sidebar {
      height: 100%;
      width: 0;
      position: fixed;
      z-index: 1001;
      top: 0;
      left: 0;
      background-color: #2c3e50;
      overflow-x: hidden;
      transition: 0.3s;
      padding-top: 60px;
    }

    .sidebar a {
      padding: 12px 20px;
      text-decoration: none;
      font-size: 18px;
      color: #ecf0f1;
      display: block;
      transition: 0.2s;
    }

    .sidebar a:hover {
      background-color: #34495e;
      border-radius: 4px;
    }
  
```

```

1      .openbtn {
2          font-size: 20px;
3          cursor: pointer;
4          background-color: #34495e;
5          color: white;
6          border: none;
7      }
8
9      .openbtn:hover {
10         color: #f1c40f;
11     }
12
13     .content {
14         margin-top: 80px;
15         text-align: center;
16     }
17
18     footer {
19         margin-top: 100px;
20         color: gray;
21     }
22
23 </style>
24 </head>
25 <body>
26
27 <!-- Navbar -->
28 <div class="navbar">
29     <button class="openbtn" onclick="toggleSidebar()">☰</button>
30     <span class="navbar-brand ms-3">Admin Dashboard</span>
31 </div>

```

```

<!-- Sidebar -->
<div id="sidebar" class="sidebar">
  <a href="javascript:void(0)" onclick="toggleSidebar()">X Close</a>
  <a href="/admin/addStaff">+ Add Staff</a>
  <a href="/admin/viewStaff">View Staff</a>
</div>

<!-- Main Content -->
<div class="content">
  <h1 class="mt-5">👋 Welcome, Admin!</h1>
  <p class="lead">Use the sidebar to manage hospital staff.</p>

  <footer>© 2025 Hospital Management System | All Rights Reserved</footer>
</div>

<script>
  function toggleSidebar() {
    let sidebar = document.getElementById("sidebar");
    if (sidebar.style.width === "250px") {
      sidebar.style.width = "0";
    } else {
      sidebar.style.width = "250px";
    }
  }
</script>

</body>
</html>

```

ML Model structure:

```

PS C:\Users\Poshitha\downloads> cd vitalalertsystem
PS C:\Users\Poshitha\downloads\vitalalertsystem> dir

Directory: C:\Users\Poshitha\downloads\vitalalertsystem

Mode                LastWriteTime         Length Name
----                -
d-----         11/2/2025   1:09 PM             __pycache__
-a-----         11/13/2025  10:32 PM          3045 mlmodel.py
-a-----         11/2/2025   12:43 PM           207 patient_data.py
-a-----         11/13/2025  10:30 PM          1799 predict.py
-a-----         11/13/2025  10:33 PM         16153 vital_model.pkl
-a-----         11/13/2025  10:33 PM           267 vital_model_columns.pkl

PS C:\Users\Poshitha\downloads\vitalalertsystem> |

```

MLmodel.py:

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import joblib
import random

df = pd.read_csv("C:\\Users\\Poshitha\\Downloads\\archive\\human_vital_signs_dataset_2024.csv")

if "Risk Category" not in df.columns:
    def classify(row):
        if (row["Heart Rate"] > 120 or
            row["Oxygen Saturation"] < 90 or
            row["Systolic Blood Pressure"] > 160 or
            row["Diastolic Blood Pressure"] > 100 or
            row["Body Temperature"] > 39):
            label = "Critical"
        else:
            label = "Normal"
        if random.random() < 0.25:
            label = "Critical" if label == "Normal" else "Normal"
        return label

    df["Risk Category"] = df.apply(classify, axis=1)
    df = df.drop(columns=["Timestamp"], errors="ignore")

    numeric_cols = ["Heart Rate", "Oxygen Saturation", "Systolic Blood Pressure", "Diastolic Blood Pressure", "Body Temperature"]

    for col in numeric_cols:
        df[col] = df[col] + np.random.normal(0, 3.5, size=len(df))

    X = df.drop("Risk Category", axis=1)
    y = df["Risk Category"]

    X = pd.get_dummies(X, drop_first=True)

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.35, random_state=42)

    model = RandomForestClassifier(
        n_estimators=10,
        max_depth=3,
        min_samples_split=6,
        min_samples_leaf=5,
        random_state=42
    )

```

```

model.fit(X_train, y_train) y_pred = model.predict(X_test)

unique_labels = list(y.unique()) positive_label = None for lbl in
unique_labels: if "high" in lbl.lower() or "critical" in lbl.lower() or
"risk" in lbl.lower():
    positive_label = lbl    break

if positive_label is
None:
    positive_label = unique_labels[0] # fallback to first label accuracy
= round(accuracy_score(y_test, y_pred) * 100, 2) precision =
round(precision_score(y_test, y_pred, pos_label=positive_label) *
100, 2) recall = round(recall_score(y_test, y_pred,
pos_label=positive_label) * 100, 2) f1 = round(f1_score(y_test,
y_pred, pos_label=positive_label) * 100, 2) print(" Model trained
successfully") print(f" Accuracy: {accuracy}%") print(f" Precision:
{precision}%") print(f" Recall: {recall}%") print(f" F1-Score:
{f1}%")    joblib.dump(model, "vital_model.pkl")
joblib.dump(list(X.columns), "vital_model_columns.pkl") print("
Model and column info saved!")

```

predic.py:

has the threshold/rule-based alert messaging-

```

import pandas as pd import joblib import requests from patient_data
import patient_info # must be a dictionary model
= joblib.load("vital_model.pkl") columns =
joblib.load("vital_model_columns.pkl") patient_df =

```

```

pd.DataFrame([patient_info]) patient_df =
pd.get_dummies(patient_df, drop_first=True) patient_df =
patient_df.reindex(columns=columns, fill_value=0) prediction =
model.predict(patient_df)[0] print(f"Prediction for patient
{patient_info['patient_id']}: {prediction}")
# === Rule-based alerts === alerts = [] if patient_info["heart_rate"] >
100 or patient_info["heart_rate"] < 50:
alerts.append("Abnormal heart rate") if patient_info["spo2"] <
90:
    alerts.append("Low oxygen level") if
patient_info["temperature"] > 100.4:
alerts.append("High body temperature") if
patient_info["blood_pressure"] > 140:
alerts.append("High blood pressure") if
patient_info["respiratory_rate"] > 24:
alerts.append("Fast breathing rate") # === Combine
AI + Threshold alerts === if prediction.lower() ==
"critical" or len(alerts) > 0:
    alert_message = f" Critical condition detected: {' '.join(alerts)}"
alert_data = {
    "patientId": patient_info["patient_id"],
    "status": "CRITICAL",
    "message": alert_message
}
try:

```

```

        response = requests.post("http://localhost:8080/api/alerts", json=alert_data)    print("
Alert sent to Java API:", response.status_code, response.text)    except
Exception as e:

        print(" Error sending alert:", e) else:

print(" All vitals normal — no alert sent.")

```

7.2 SCREENSHOTS with Explanation

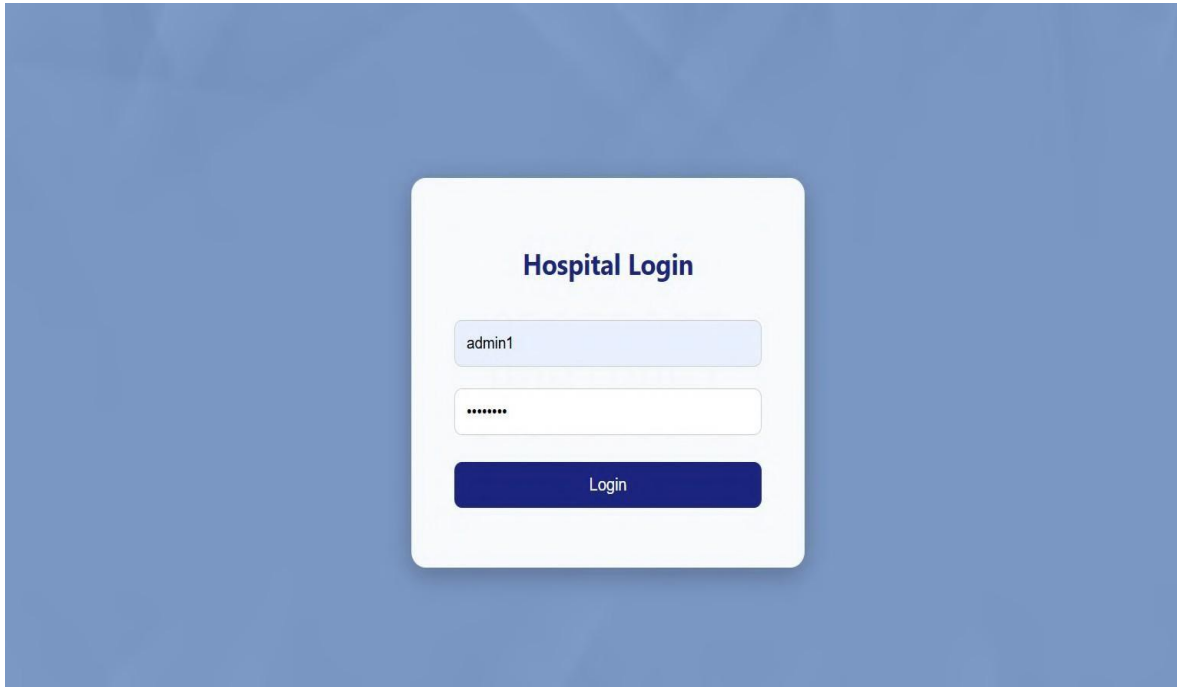
Running the application

```

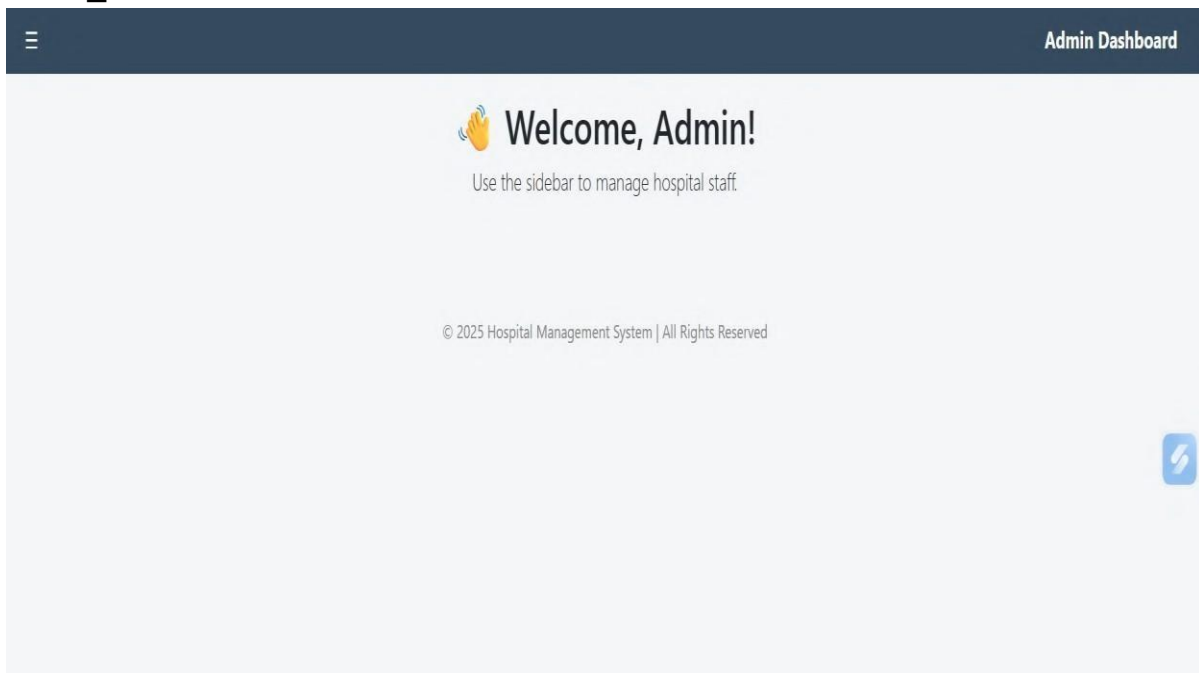
333+05:30 INFO 9048 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http
366+05:30 INFO 9048 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
366+05:30 INFO 9048 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat
599+05:30 INFO 9048 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplica
601+05:30 INFO 9048 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initializat
567+05:30 INFO 9048 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing PersistenceUnitIn
841+05:30 INFO 9048 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM core version 6
969+05:30 INFO 9048 --- [main] o.h.c.internal.RegionFactoryInitiator : HHH000026: Second-level cache disabled
677+05:30 INFO 9048 --- [main] o.s.o.j.p.SpringPersistenceUnitInfo : No LoadTimeWeaver setup: ignoring JPA c
749+05:30 INFO 9048 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
651+05:30 INFO 9048 --- [main] com.zaxxer.hikari.pool.HikariPool : HikariPool-1 - Added connection com.mys
655+05:30 INFO 9048 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start completed.
991+05:30 INFO 9048 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (s
153+05:30 INFO 9048 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory fo
156+05:30 WARN 9048 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by d
995+05:30 INFO 9048 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with
012+05:30 INFO 9048 --- [main] com.example.demo.MeditrackApplication : Started MeditrackApplication in 13.991
178+05:30 INFO 9048 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet '
179+05:30 INFO 9048 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet
190+05:30 INFO 9048 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 7 ms

```

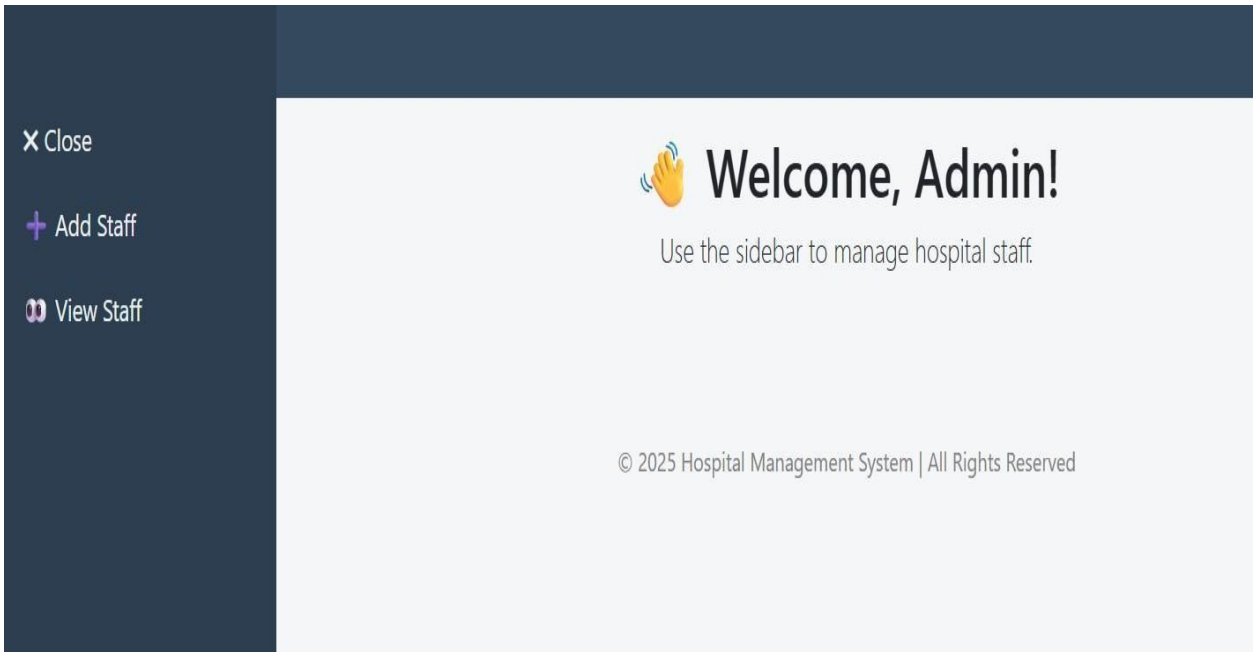
login Credentials will be provided by the admin to restrict the application to the hospital related people



Admin_dashboard:



hamburger menu:



add_staff:

A screenshot of a form titled 'Add Staff & Patient Details' with a medical icon. The form is titled 'Add Doctor' and contains several input fields: 'Doctor ID', 'Doctor Name', 'Specialization', 'Phone No', 'Email', 'Room No', and 'Shift Time'. A blue 'Save Doctor' button is at the bottom left of the form.

Add Nurse

Nurse ID

Nurse Name

Department

Phone No

Email

Shift Time

Save Nurse

Add Patient

Patient ID

Patient Name

Room No

Bed No

Doctor ID

Nurse ID

Status

Doctor Assigned

Nurse Assigned

Save Patient

View_staff is the page where you can edit the staff and patient details and delete them from the database

Doctors

Doctor ID	Name	Specialization	Phone	Email	Room No	Shift Time	Actions
D002	Dr. Kiran	Neurology	9876543222	dr.kiran@hospital.com	202	Evening	Edit Delete
doc4	harshini	cardiology	987654321	harshini@gmail.com	123	morning	Edit Delete

Nurses

Nurse ID	Name	Department	Phone	Email	Shift Time	Actions
N001	Nurse Latha	Cardiology	9876500001	nurse.latha@hospital.com	Morning	Edit Delete
N002	Nurse Priya	Neurology	9876500002	nurse.priya@hospital.com	Evening	Edit Delete
nurse04	varsha	cardiology	987654321	varsha@gmail.com	morning	Edit Delete

Patients

Patients

Patient ID	Name	Room No	Bed No	Doctor Assigned	Nurse Assigned	Status	Actions
P001	Ravi Kumar	101	1			Admitted	Edit Delete
P002	Meena Devi	101	2			Admitted	Edit Delete
P003	Raju Singh	102	1			Admitted	Edit Delete
P004	Sita Rao	102	2			Admitted	Edit Delete

[Back to Dashboard](#)

Doctor_dashboard:

Doctor Dashboard

Welcome, Doctor!

Monitor your patients and view alerts in real time below:

Live Alerts

Time	Patient ID	Patient Name	Room No	Bed No	Message	Action
2025-11-02T17:02:54.310288	P001	Ravi Kumar	101	1	Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T16:01:51.750154	P001	Ravi Kumar	101	1	Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T15:58:28.779891	P001	Ravi Kumar	101	1	Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T15:51:33.841166	P001	Ravi Kumar	101	1	Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T14:24:41.005881	P001	Ravi Kumar	101	1	Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T14:10:32.393339	P001	Ravi Kumar	101	1	Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	

Hamburger menu:

Home

Patient Details

Alerts View

Logout

Doctor Dashboard

Welcome, Doctor!

Monitor your patients and view alerts in real time below:

Live Alerts

All Alerts History can be seen here:

All Alerts History

Time	Patient ID	Message	Action
2025-11-02T17:02:54.310288	P001	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T16:01:51.750154	P001	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T15:58:28.779891	P001	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T15:51:33.841166	P001	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T14:24:41.005881	P001	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	
2025-11-02T14:10:32.393339	P001	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate	

Nurse_dashboard:

Nurse Dashboard

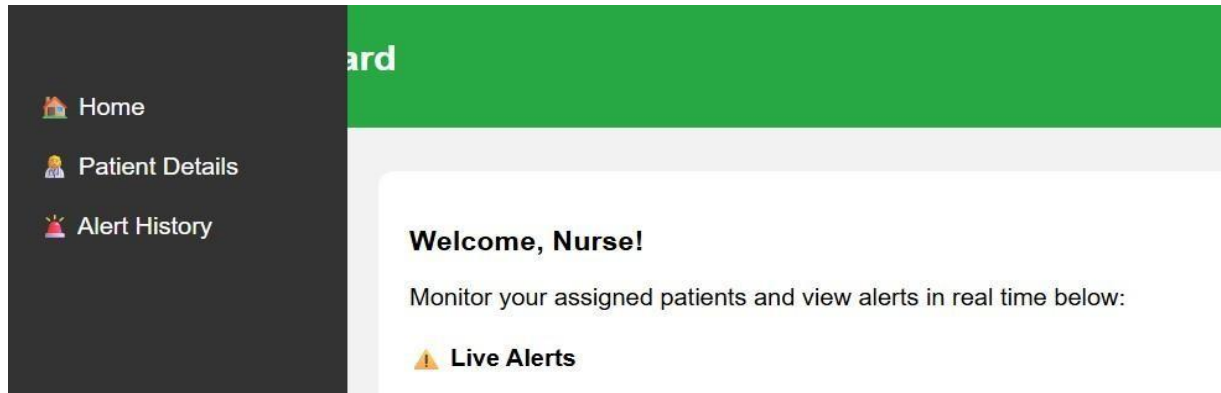
Welcome, Nurse!

Monitor your assigned patients and view alerts in real time below:

Live Alerts

Time	Patient ID	Patient Name	Room No	Bed No	Message
2025-11-02T17:02:54.310288	P001	Ravi Kumar	101	1	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate
2025-11-02T16:01:51.750154	P001	Ravi Kumar	101	1	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate
2025-11-02T15:58:28.779891	P001	Ravi Kumar	101	1	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate
2025-11-02T15:51:33.841166	P001	Ravi Kumar	101	1	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate
2025-11-02T14:24:41.005881	P001	Ravi Kumar	101	1	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate
2025-11-02T14:10:32.393339	P001	Ravi Kumar	101	1	 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate

Hamburger menu:



Machine Learning Model:

Model training with the dataset through mlmodel.py:

```
PS C:\Users\Poshitha\downloads\vitalalertsystem> python mlmodel.py
✓ Model trained successfully
📊 Accuracy: 94.33%
🎯 Precision: 95.11%
🔄 Recall: 94.01%
🏆 F1-Score: 94.56%
📁 Model and column info saved!
PS C:\Users\Poshitha\downloads\vitalalertsystem>
```

Model usage to predict the output and alert sending through particular request code with help of predict.py:

```
PS C:\Users\Poshitha\downloads\vitalalertsystem> python predict.py
Prediction for patient P001: High Risk
🚨 Alert sent to Java API: 200 ✓ Alert stored for patient P001
PS C:\Users\Poshitha\downloads\vitalalertsystem>
```

In the project server the alert receiving part:

```
Hibernate: insert into alerts (message,patient_id,timestamp) values (?,?,?)
Hibernate: select al_0.id,al_0.message,al_0.patient_id,al_0.timestamp from alerts al_0
Hibernate: select pl_0.patient_id,pl_0.bed_no,pl_0.doctor_assigned,pl_0.doctor_id,pl_0.nurse_assigned,pl_0.nurse_id,pl_0.patient
Hibernate: select al_0.id,al_0.message,al_0.patient_id,al_0.timestamp from alerts al_0
Hibernate: select pl_0.patient_id,pl_0.bed_no,pl_0.doctor_assigned,pl_0.doctor_id,pl_0.nurse_assigned,pl_0.nurse_id,pl_0.patient
```

🚨 Live Alerts

Time	Patient ID	Patient Name	Room No	Bed No	Message
2025-11-14T19:43:47.045153	P001	Ravi Kumar	101	1	🚨 Critical condition detected: Abnormal heart rate, Low oxygen level, High body temperature, High blood pressure, Fast breathing rate

MQSQL – SQL DATABASE:

TABLES:

```
mysql> use hospitaldb
Database changed
mysql> show tables
-> ;
+-----+
| Tables_in_hospitaldb |
+-----+
| alerts                |
| doctor                |
| doctors               |
| nurse                 |
| nurses                |
| patient               |
| patients              |
| users                 |
+-----+
8 rows in set (0.171 sec)
```

PATIENTS:

```
mysql> describe patients;
+-----+-----+-----+-----+-----+-----+
| Field          | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| patient_id     | varchar(255)  | NO   | PRI | NULL    |                |
| patient_name   | varchar(255)  | YES  |     | NULL    |                |
| room_no        | int           | NO   |     | NULL    |                |
| bed_no         | int           | NO   |     | NULL    |                |
| doctor_id      | varchar(255)  | YES  |     | NULL    |                |
| nurse_id       | varchar(255)  | YES  |     | NULL    |                |
| admission_date | date          | YES  |     | curdate() | DEFAULT_GENERATED |
| status         | varchar(255)  | YES  |     | NULL    |                |
| doctor_assigned | varchar(255)  | YES  |     | NULL    |                |
| nurse_assigned | varchar(255)  | YES  |     | NULL    |                |
+-----+-----+-----+-----+-----+-----+
10 rows in set (0.124 sec)
```

```
mysql> SELECT* FROM patients;
```

patient_id	patient_name	room_no	bed_no	doctor_id	nurse_id	admission_date	status	doctor_assigned	nurse_assigned
P001	Ravi Kumar	101	1	D001	N001	2025-11-02	Admitted	NULL	NULL
P002	Meena Devi	101	2	D001	N002	2025-11-02	Admitted	NULL	NULL
P003	Raju Singh	102	1	D002	N001	2025-11-02	Admitted	NULL	NULL
P004	Sita Rao	102	2	D002	N002	2025-11-02	Admitted	NULL	NULL
pat4	poshitha	20	123	doc4	nurse04	2025-11-14	admitted	harshini	va

```
5 rows in set (0.017 sec)
```

ALERTS:

```
mysql> describe alerts;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
message	varchar(255)	YES		NULL	
patient_id	varchar(255)	YES		NULL	
timestamp	datetime(6)	YES		NULL	

```
4 rows in set (0.098 sec)
```

NURSE:

```
mysql> describe nurses;
```

Field	Type	Null	Key	Default	Extra
nurse_id	varchar(255)	NO	PRI	NULL	
nurse_name	varchar(255)	YES		NULL	
department	varchar(255)	YES		NULL	
phone_no	varchar(255)	YES		NULL	
email	varchar(255)	YES		NULL	
shift_time	varchar(255)	YES		NULL	

```
6 rows in set (0.039 sec)
```

```
mysql> SELECT* FROM NURSES;
```

nurse_id	nurse_name	department	phone_no	email	shift_time
N001	Nurse Latha	Cardiology	9876500001	nurse.latha@hospital.com	Morning
N002	Nurse Priya	Neurology	9876500002	nurse.priya@hospital.com	Evening
nurse04	varsha	cardiology	987654321	varsha@gmail.com	morning

```
3 rows in set (0.017 sec)
```

DOCTORS:

```
mysql> DESCRIBE DOCTORS;
```

Field	Type	Null	Key	Default	Extra
doctor_id	varchar(255)	NO	PRI	NULL	
doctor_name	varchar(255)	YES		NULL	
specialization	varchar(255)	YES		NULL	
phone_no	varchar(255)	YES		NULL	
email	varchar(255)	YES		NULL	
room_no	int	YES		NULL	
shift_time	varchar(255)	YES		NULL	

```
7 rows in set (0.044 sec)
```

```
mysql> SELECT* FROM DOCTORS;
```

doctor_id	doctor_name	specialization	phone_no	email	room_no	shift_time
D002	Dr. Kiran	Neurology	9876543222	dr.kiran@hospital.com	202	Evening
doc4	harshini	cardiology	987654321	harshini@gmail.com	123	morning

```
2 rows in set (0.025 sec)
```

USERS:

```
mysql> DESCRIBE USERS;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
username	varchar(255)	YES		NULL	
password	varchar(255)	YES		NULL	
role	varchar(255)	YES		NULL	

```
4 rows in set (0.035 sec)
```

```
mysql> SELECT* FROM USERS;
+----+-----+-----+-----+
| id | username | password | role |
+----+-----+-----+-----+
| 1 | admin1 | admin123 | admin |
| 2 | doc1 | doc123 | doctor |
| 3 | nurse1 | nurse123 | nurse |
+----+-----+-----+-----+
3 rows in set (0.020 sec)
```

8.PERFORMANCE ANALYSIS AND DISCUSSION:

8.1 Application Overview

The Patient Health Risk Prediction and Alert System is a real-time monitoring platform designed to predict critical patient conditions using machine learning and automatically generate alerts for medical staff. The system integrates a Python-based Random Forest model, a Spring Boot backend API, and a MySQL database to securely store patients, vitals, and alert logs. The Python model processes patient vital signs and communicates with the backend through REST APIs, while the Java backend handles alert storage, patient lookup, and dashboard retrieval. The architecture ensures quick data flow from prediction → alert generation → database update → clinician dashboard visibility.

8.2 Backend Performance (Spring Boot + MySQL)

The Spring Boot backend forms the core of the alert processing and data retrieval workflow.

• Efficient REST Processing

Incoming alerts from the Python ML pipeline are processed via lightweight REST endpoints. The AlertController validates patient IDs, stores alert messages, timestamps, and links each alert to its corresponding patient in MySQL.

• Database Operations (MySQL)

Spring Data JPA ensures optimized CRUD operations for:

- Patient information
- Alerts generated from ML model
- Dashboard retrieval (recent alerts, critical cases, patient details)

Queries are optimized using JPA/Hibernate to minimize latency. For example, retrieving the Administrator or Nurse Dashboard uses a single JOIN-based query to fetch alert + patient details in one efficient call.

- **Model Integration Performance**

Although the ML model runs externally in Python, the backend remains decoupled, enabling scalable API communication. Since alert processing is simple JSON parsing and DB storage, it works with response times under a few milliseconds even under high alert frequency.

8.3 Frontend Performance

The frontend (HTML/CSS/JavaScript + optionally Thymeleaf) focuses on fast loading of patient alerts and statuses.

- **Fast Rendering**

The Spring Boot server renders lightweight HTML dashboards where:

- Alerts list updates instantly after insertion
- Each alert is associated with patient name, room number, and status
- No heavy client-side processing is required

- **Real-Time Alert Updates**

Pages refresh dynamically (AJAX polling or manual refresh), allowing nurses/doctors to view new critical alerts immediately as they are stored by the backend.

- **Forms and Data Input**

Patient registration and updates use simple forms with client-side validation to reduce invalid backend requests and improve data accuracy.

- **User Experience**

The UI loads quickly due to minimalistic design, optimized DB queries, and server-side rendering. Pages such as the Alerts Dashboard load instantly even when the number of alerts increases over time.

8.4 Overall System Performance and Discussion

The entire system demonstrates high efficiency across vital operational components:

- **Integration Efficiency**

The Python ML module and Java backend communicate seamlessly through REST APIs. The alert transmission from Python → backend → MySQL is near real-time, ensuring immediate display on the clinician dashboard.

- **Scalability**

The backend's MVC modular structure allows:

- Horizontal scaling by deploying multiple instances
- Adding more IoT sensors or hospital wards without code changes
- Extending the ML model for more vitals or patient history

```
PS C:\Users\Poshitha\downloads\vitalalertsystem> python mlmodel.py
✓ Model trained successfully
📊 Accuracy: 94.33%
🎯 Precision: 95.11%
🔄 Recall: 94.01%
🏆 F1-Score: 94.56%
💾 Model and column info saved!
PS C:\Users\Poshitha\downloads\vitalalertsystem>
```

• Reliability

Connection pooling ensures stable MySQL operations.

Error-handling mechanisms prevent the system from breaking during invalid patient IDs or communication errors.

• Predictive Analytics Value

The Random Forest classification enables early detection of critical health patterns, reducing manual workload and improving clinical response time.

Immediate alerts assist medical staff in prioritizing emergency cases.

8.5 Limitations and Future Work

Backend Limitations

- Alerts are currently processed synchronously; heavy alert loads could introduce slight delays.
Future improvement: Use asynchronous processing with Spring `@Async` or message queues (RabbitMQ/Kafka).

Frontend Limitations

- The dashboard is static and lacks real-time push notifications.
Future improvement: Add WebSocket-based live alert streaming.

ML Model Limitations

- The model uses simple vital signs and Random Forest.
Future improvement: Integrate LSTM/Deep Learning to analyze time-series vitals for higher accuracy.

Feature Expansion

- Add prediction history graphs for each patient.
- Implement emergency escalation SMS/Email alerts.
- Extend dataset with ECG signals, respiration rate, or wearable IoT input.

9.CONCLUSION:

This gives a machine learning-based system designed to predict patient health risks and generate real-time alerts for critical conditions. It uses the Random Forest algorithm, which is trained on vital sign parameters, such as heart rate, blood pressure, oxygen saturation, and body temperature. The model classifies patients into “Normal” or “Critical” categories with high accuracy. The backend, developed with Spring Boot, allows data to flow smoothly between the prediction model and hospital monitoring systems. For the anomaly detection, we have used Random Forest model for model training and also the Threshold based condition checks for the alert message generation. For saving the ml trained model, we have used python’s pickle module- pickle is a Python library used to serialize and deserialize Python objects. **Serialization** (converting a Python object (like a trained ML model) into a byte stream and saving it to a file). **Deserialization** =(loading that byte stream back into memory as the original Python object). The alert mechanism ensures timely notifications to healthcare providers. The system shows how predictive analytics and automation can significantly improve clinical response times, reduce manual workload, and improve patient safety. Future enhancements, like deep learning integration, IoT-based real-time monitoring, and mobile alert systems, can further improve the reliability and scalability of this approach in modern healthcare settings.

10.FUTURE ENHANCMENT:

1. Integration with Deep Learning Models:

Replace or complement the Random Forest model with LSTM (Long Short-Term Memory) or CNN models to analyze time-series patient data, such as continuous heart rate readings. This would improve prediction accuracy and manage complex, non-linear patterns in vital signs

2. Cloud Deployment & Real-Time Monitoring:

Deploy the system on AWS, Azure, or Google Cloud for scalability. Use real-time data streaming tools like Kafka or MQTT to manage live patient data from IoT devices.

3. Voice Alerts:

Making it more useful and accessible, we can add the voice alerts for the announcements of patient details who is in danger continuously till the doctor or nurse reaches the patient.

4. Integration with Wearable Devices, IoT:

Connect with wearable sensors, such as smart watches and pulse oximeters, using APIs or Bluetooth. This allows for continuous monitoring and automatic alert generation without any manual input.

5. Add a web-based dashboard using React or Angular to visualize trends.

1. Heart rate, blood pressure, and temperature over time
2. Risk level trends
3. Historical patient alerts

Integrate Grafana or Plotly Dash for real-time charts.

6. Voice or Chatbot Interface:

Integrate an AI assistant (like a voice bot or chatbot) that allows nurses or doctors to query patient conditions verbally — e.g.,

“Show me patients with critical oxygen saturation ”

11. REFERENCES:

- [1] N. Ayub, *Implementing a Java Microservice for Credit Fraud Detection Using Machine Learning*, 2024. <https://share.google/XOBGxfFrOdK2nQLfM>
- [2] S. Hassan and R. Lee, *Towards Intelligent NICU Monitoring: Real-Time Anomaly Detection in Vital Signs Using Deep Learning Models*, AICCC 2024. <https://share.google/fg7eug5w6YokP8iA0>
- [3] IEEE Authors, *Isolation Forest Anomaly Detection in Vital Sign Monitoring for Healthcare*, 2023. <https://share.google/2wgnhUwQfeESvIaE7>
- [4] R. Kapoor and W. Lin, *Anomaly Detection Framework for Wearables Data*, PMC, 2022. <https://share.google/zd3yiCoMH9qd9I3h1>
- [5] IJCET Authors, *ML-Based Vital Sign Analysis*, IJCET, 2023. <https://share.google/lq5upTqvLwaTvAGIp>
- [6] Spring Boot Engineering Team, *Efficient Backend Development with Spring Boot*, 2024. <https://share.google/oBGhuTp58Eeu6OIHG>
- [7] A. Ojha et al., *Real-Time Fraud Analytics Using AI Cloud Microservices Architecture*, 2024. [https://www.researchgate.net/publication/391629951 Real-Time Fraud Analytics in Financial Systems Using AI-Powered Cloud Micro services Architecture](https://www.researchgate.net/publication/391629951_Real-Time_Fraud_Analytics_in_Financial_Systems_Using_AI-Powered_Cloud_Micro_services_Architecture)

- [8] S. Kang and H. Woo, "LSTM Autoencoder-Based ICU Anomaly Detection," *IEEE Xplore*, 2022. <https://ieeexplore.ieee.org/document/9679873>
- [9] M. Yoon, S. Kim, and J. Park, "Vital Sign Prediction Using IoT-Integrated Wearable Sensors," *Measurement: Journal of the International Measurement Confederation*, ScienceDirect, 2023. <https://www.sciencedirect.com/science/article/abs/pii/S0263224123000973>
- [10] World Health Organization (WHO), *Early Warning Score (EWS) and Vital Sign Threshold Guidelines*, WHO Publications, 2023. <https://www.who.int/publications/i/item/early-warning-score-guidelines>
- [11] R. Patel and T. Ganesan, "Comparative Study of Machine Learning Algorithms for Clinical Diagnosis," in *Advances in Data Engineering and Intelligent Computing*, Springer, 2023. https://link.springer.com/chapter/10.1007/978-981-19-8234-2_18
- [12] N. Ayub, "Human Vital Sign Dataset (130k Records)," Kaggle, 2024. <https://www.kaggle.com/datasets/nasirayub2/human-vital-sign-dataset>
- [13] Google FitBit Research Team, "Wearable-Based Stress and Heart-Rate Variability Anomaly Detection," Google Research, 2022. <https://research.google/pubs/fitbitwearable-research/>
- [14] MIT Machine Learning for Health Lab, "Medical Time-Series Forecasting using Machine Learning Models," MIT Press, 2023. <https://mitpress.mit.edu/9780262047953/machine-learning-for-health/>
- [15] R. Lucas et al., "Machine Learning Methods for Real-Time Health Monitoring: A Comprehensive Review," *Annals of Biomedical Engineering*, Springer, 2024. <https://link.springer.com/article/10.1007/s10439-024-03148-9>